Chapter 5 Loops

Motivations

Suppose that you need to print a string (e.g., "Welcome to Java!") a hundred times. It would be tedious to have to write the following statement a hundred times:

System.out.println("Welcome to Java!");

So, how do you solve this problem?

Opening Problem

Problem:

```
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```

100 times

```
...
...
...
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```


Loops

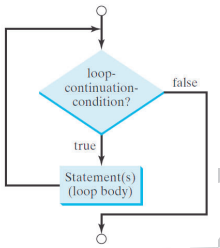
- *Loops* are constructs that control repeated executions of a block of statements.
- The concept of looping is fundamental to programming.
- Java provides three types of loop statements:
  - **while** loops,
  - **do-while** loops,
  - **for** loops.

while Loop Flow Chart

A **while** loop executes statements repeatedly while the condition is true.

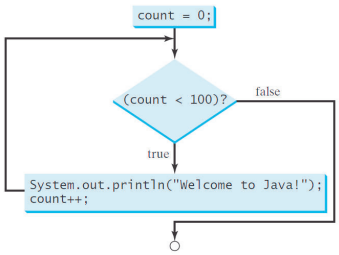
Syntax:

```
while (loop-continuation-condition) {
  // loop-body;
  Statement(s);
}
```



Introducing while Loops

```
int count = 0;
while (count < 100) {
  System.out.println("Welcome to Java");
  count++;
}
```



animation

Trace while Loop

Initialize count

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```


animation

Trace while Loop, cont.

(count < 2) is true

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Print Welcome to Java

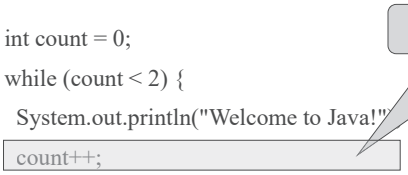


animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Increase count by 1
count is 1 now



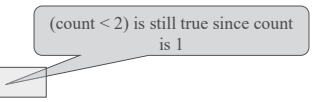


animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

(count < 2) is still true since count is 1





animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Print Welcome to Java



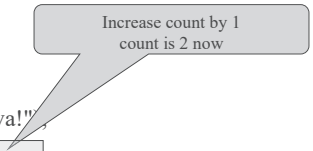


animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

Increase count by 1
count is 2 now





animation

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

(count < 2) is false since count is 2 now



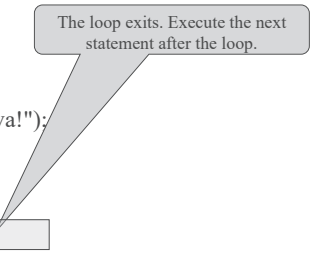


animation

Trace while Loop

```
int count = 0;
while (count < 2) {
  System.out.println("Welcome to Java!");
  count++;
}
```

The loop exits. Execute the next statement after the loop.





Example

```
int sum = 0, i = 1;
while (i < 5) {
    sum = sum + i;
    i++;
}
System.out.println("sum is " + sum);
```



Example

```
int sum = 0, i = 1;
while (i < 10) {
    sum = sum + i;
}
```


Problem: Repeat Addition Until Correct

Recall that Listing 3.1 AdditionQuiz.java gives a program that prompts the user to enter an answer for a question on addition of two single digits. Using a loop, you can now rewrite the program to let the user enter a new answer until it is correct.

RepeatAdditionQuiz Run

Problem: Guessing Numbers

Write a program that randomly generates an integer between 0 and 100, inclusive. The program prompts the user to enter a number continuously until the number matches the randomly generated number. For each user input, the program tells the user whether the input is too low or too high, so the user can choose the next input intelligently. Here is a sample run:

GuessNumberOneTime Run

GuessNumber Run

Problem: An Advanced Math Learning Tool

The Math subtraction learning tool program generates just one question for each run. You can use a loop to generate questions repeatedly. This example gives a program that generates five questions and reports the number of the correct answers after a student answers all five questions.

SubtractionQuizLoop Run

Ending a Loop with a Sentinel Value

Often the number of times a loop is executed is not predetermined. You may use an input value to signify the end of the loop. Such a value is known as a *sentinel value*.

Write a program that reads and calculates the sum of an unspecified number of integers. The input 0 signifies the end of the input.

SentinelValue Run

Caution

Do not use floating-point values for equality checking in a loop control. Since floating-point values are approximations for some values, using them could result in imprecise counter values and inaccurate results. Consider the following code for computing 1 + 0.9 + 0.8 + ... + 0.1:

```
double item = 1; double sum = 0;
while (item != 0) { // No guarantee item will be 0
  sum += item;
  item -= 0.1;
}
System.out.println(sum);
```


Example

- How many times are the following loop bodies repeated? What is the output of each loop?

```
int i = 1;
while (i < 10)
  if (i % 2 == 0)
    System.out.println(i);
```

(a)

```
int i = 1;
while (i < 10)
  if (i % 2 == 0)
    System.out.println(i++);
```

(b)

```
int i = 1;
while (i < 10)
  if ((i++) % 2 == 0)
    System.out.println(i);
```

(c)

Example

Suppose the input is 2 3 4 5 0. What is the output of the following code?

```
import java.util.Scanner;

public class Test {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

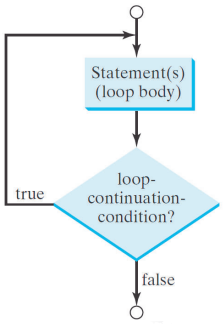
    int number, max;
    number = input.nextInt();
    max = number;

    while (number != 0) {
      number = input.nextInt();
      if (number > max)
        max = number;
    }

    System.out.println("max is " + max);
    System.out.println("number " + number);
  }
}
```

do-while Loop

A **do-while** loop is the same as a while loop except that it executes the loop body first and then checks the loop continuation condition.



Syntax:

```
do {
  // Loop body;
  Statement(s);
} while (loop-continuation-condition);
```


Ending a Loop with a Sentinel Value

- Rewrite the program that reads and calculates the sum of an unspecified number of integers using do-while. The input 0 signifies the end of the input.

TestDoWhile

Caution, cont.

The following loop is wrong:

```
int i=0;
while (i < 10); ← Logic Error
{
  System.out.println("i is " + i);
  i++;
}
```

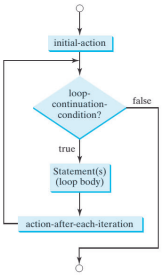
In the case of the do loop, the following semicolon is needed to end the loop.

```
int i=0;
do {
  System.out.println("i is " + i);
  i++;
} while (i<10); ← Correct
```


for Loops

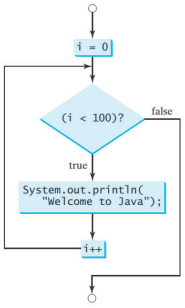
A **for** loop has a concise syntax for writing loops

```
for (initial-action; loop-continuation-condition; action-after-each-iteration) {
  // loop body;
  Statement(s);
}
```



for Loops

```
int i;
for (i = 0; i < 100; i++) {
  System.out.println("Welcome to Java!");
}
```



animation

Trace for Loop

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println(
    "Welcome to Java!");
}
```

Declare i

animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println(
    "Welcome to Java!");
}
```

Execute initializer
i is now 0

animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println( "Welcome to Java!");
}
```

(i < 2) is true
since i is 0

animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Print Welcome to Java



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Execute adjustment statement
i now is 1



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

(i < 2) is still true
since i is 1



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Print Welcome to Java



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Execute adjustment statement
i now is 2



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

(i < 2) is false
since i is 2



animation

Trace for Loop, cont.

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

Exit the loop. Execute the next
statement after the loop



Note

```
int i;
for (i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```

```
for (int i = 0; i < 2; i++) {
  System.out.println("Welcome to Java!");
}
```



Note

The initial-action in a for loop can be a list of zero or more comma-separated expressions. The action-after-each-iteration in a for loop can be a list of zero or more comma-separated statements. Therefore, the following two for loops are correct. They are rarely used in practice, however.

```
for (int i = 1; i < 100; System.out.println(i++));

for (int i = 0, j = 0; (i + j < 10); i++, j++) {
  // Do something
}
```



Note

If the loop-continuation-condition in a for loop is omitted, it is implicitly true. Thus the statement given below in (a), which is an infinite loop, is correct. Nevertheless, it is better to use the equivalent loop in (b) to avoid confusion:

```
for ( ; ; ) {
  // Do something
}
```
(a)

Equivalent

```
while (true) {
  // Do something
}
```
(b)



Caution

Adding a semicolon at the end of the for clause before the loop body is a common mistake, as shown below:

Error

```
for (int i = 0; i < 10; i++);
{
  System.out.println("i is " + i);
}
```
(a)

Empty body

```
for (int i = 0; i < 10; i++) { };
{
  System.out.println("i is " + i);
}
```
(b)



Example

Do the following two loops result in the same value in **sum**?

```
for (int i = 0; i < 10; ++i) {
  sum += i;
}
```
(a)

```
for (int i = 0; i < 10; i++) {
  sum += i;
}
```
(b)



Example

Count the number of iterations in the following loops.

```
int count = 0;
while (count < n) {
  count++;
}
```
(a)

```
for (int count = 0;
  count <= n; count++) {
}
```
(b)

```
int count = 5;
while (count < n) {
  count++;
}
```
(c)



Which Loop to Use?

The three forms of loop statements, while, do-while, and for, are expressively equivalent; that is, you can write a loop in any of these three forms. For example, a while loop in (a) in the following figure can always be converted into the following for loop in (b):

```
while (loop-continuation-condition) {
  // Loop body
}
```
(a)

Equivalent

```
for ( ; loop-continuation-condition; )
  // Loop body
}
```
(b)



Which Loop to Use?

A for loop in (a) in the following figure can generally be converted into the following while loop in (b) except in certain special :

```
for (initial-action;
     loop-continuation-condition;
     action-after-each-iteration) {
  // Loop body;
}
```
(a)

Equivalent

```
initial-action;
while (loop-continuation-condition) {
  // Loop body;
  action-after-each-iteration;
}
```
(b)



Recommendations

Use the one that is most intuitive and comfortable for you.

- A *for loop* may be used if the number of repetitions is known, as, for example, when you need to print a message 100 times.
- A *while loop* may be used if the number of repetitions is not known, as in the case of reading the numbers until the input is 0.
- A *do-while loop* can be used to replace a while loop if the loop body has to be executed before testing the continuation condition.



Nested Loops

- *A loop can be nested inside another loop.*
- *Nested loops* consist of an outer loop and one or more inner loops.
- Each time the outer loop is repeated, the inner loops are reentered, and started anew.

Nested Loops

Problem: Write a program that uses nested for loops to print a multiplication table.

```
                Multiplication Table
     1   2   3   4   5   6   7   8   9
--------------------------------------------
1 |  1   2   3   4   5   6   7   8   9
2 |  2   4   6   8  10  12  14  16  18
3 |  3   6   9  12  15  18  21  24  27
4 |  4   8  12  16  20  24  28  32  36
5 |  5  10  15  20  25  30  35  40  45
6 |  6  12  18  24  30  36  42  48  54
7 |  7  14  21  28  35  42  49  56  63
8 |  8  16  24  32  40  48  56  64  72
9 |  9  18  27  36  45  54  63  72  81
```

MultiplicationTable

Run

Example

Show the output of the following programs.
(Hint: Draw a table and list the variables in the columns to trace these programs.)

```
public class Test {
  public static void main(String[] args) {
    for (int i = 1; i < 5; i++) {
      int j = 0;
      while (j < i) {
        System.out.print(j + " ");
        j++;
      }
    }
  }
}
```

(a)

```
public class Test {
  public static void main(String[] args) {
    int i = 0;
    while (i < 5) {
      for (int j = i; j > 1; j--)
        System.out.print(j + " ");
      System.out.println("****");
      i++;
    }
  }
}
```

(b)

Example (cont.)

```
public class Test {
  public static void main(String[] args) {
    int i = 1;
    do {
      int num = 1;
      for (int j = 1; j <= i; j++) {
        System.out.print(num + "G");
        num += 2;
      }

      System.out.println();
      i++;
    } while (i <= 5);
  }
}
```


Problem: Finding the Greatest Common Divisor

Problem: Write a program that prompts the user to enter two positive integers and finds their greatest common divisor.

Solution: Suppose you enter two integers 4 and 2, their greatest common divisor is 2. Suppose you enter two integers 16 and 24, their greatest common divisor is 8. So, how do you find the greatest common divisor? Let the two input integers be n1 and n2. You know number 1 is a common divisor, but it may not be the greatest commons divisor. So you can check whether k (for k = 2, 3, 4, and so on) is a common divisor for n1 and n2, until k is greater than n1 or n2.

GreatestCommonDivisor

Run

Problem: Predicting the Future Tuition

Problem: Suppose that the tuition for a university is $10,000 this year and tuition increases 7% every year. In how many years will the tuition be doubled?

FutureTuition

Run

Problem: Predicating the Future Tuition

```
public class FutureTuition {
  public static void main(String[] args) {
    double tuition = 10000;   // Year 0
    int year = 0;
    while (tuition < 20000) {
      tuition = tuition * 1.07;
      year++;
    }

    System.out.println("Tuition will be doubled in " +
      year + " years");
    System.out.printf("Tuition will be $%.2f in %1d years",
      tuition, year);
  }
}
```


Case Study: *Converting Decimals to Hexadecimals*

Hexadecimals are often used in computer systems programming (see Appendix F for an introduction to number systems). How do you convert a decimal number to a hexadecimal number? To convert a decimal number d to a hexadecimal number is to find the hexadecimal digits h_n, h_{n-1}, h_{n-2}, ... , h_2, h_1, and h_0 such that

$$d = h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + ... + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

These hexadecimal digits can be found by successively dividing d by 16 until the quotient is 0. The remainders are h_0, h_1, h_2, ... , h_{n-2}, h_{n-1}, and h_n.

Dec2Hex

Run

Using `break` and `continue`

The **break** and **continue** keywords provide additional controls in a loop.



`break`

You have used the keyword **break** in a **switch** statement. You can also use **break** in a loop to immediately terminate the loop.



`break`

```
public class TestBreak {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      sum += number;
      if (sum >= 100)
        break;
    }

    System.out.println("The number is " + number);
    System.out.println("The sum is " + sum);
  }
}
```



`continue`

When it is encountered, it ends the current iteration and program control goes to the end of the loop body .

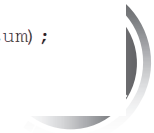


`continue`

```
public class TestContinue {
  public static void main(String[] args) {
    int sum = 0;
    int number = 0;

    while (number < 20) {
      number++;
      if (number == 10 || number == 11)
        continue;
      sum += number;
    }

    System.out.println("The sum is " + sum);
  }
}
```



Using `break` and `continue`

Examples for using the `break` and `continue` keywords:

- TestBreak.java

TestBreak Run

- TestContinue.java

TestContinue Run



Problem: Displaying Prime Numbers

Problem: Write a program that displays the first 50 prime numbers in five lines, each of which contains 10 numbers. An integer greater than 1 is *prime* if its only positive divisor is 1 or itself. For example, 2, 3, 5, and 7 are prime numbers, but 4, 6, 8, and 9 are not.

Solution: The problem can be broken into the following tasks:

- For number = 2, 3, 4, 5, 6, ..., test whether the number is prime.
- Determine whether a given number is prime.
- Count the prime numbers.
- Print each prime number, and print 10 numbers per line.

PrimeNumber Run

